

ETNAir — Documentation complète

Plateforme de location de logements étudiants — projet ETNA C2W-CB11

Équipe : Harvey Mouloundou, Nelson Pires Da Silva, Alexandre Chikhaoui, Yamine Ikhlef

Table des matières

1. Présentation du projet
 2. Architecture technique générale
 3. Frontend Vue 3
 4. Backend Express et API REST
 5. Base de données PostgreSQL et Prisma
 6. Conteneurisation avec Docker
 7. Déploiement Kubernetes
 8. Stratégie de tests
 9. Pipeline CI/CD GitLab
 10. Démarrage rapide et commandes utiles
-

1. Présentation du projet

ETNAir est une plateforme web inspirée d'Airbnb mais pensée spécifiquement pour les étudiants. Elle met en relation deux profils d'utilisateurs aux besoins complémentaires. D'un côté, les locataires peuvent rechercher des logements en filtrant par ville, par prix ou par dates de disponibilité, consulter les fiches détaillées avec photos et avis, puis réserver directement en ligne sans intermédiaire. De l'autre, les propriétaires publient leurs annonces, gèrent leurs photos, ajustent leurs prix selon la période, et acceptent ou refusent les demandes de réservation qu'ils reçoivent.

Au-delà de la mise en relation, la plateforme propose plusieurs fonctionnalités qui enrichissent l'expérience utilisateur. Le système de favoris permet à un visiteur de mettre de côté les annonces qui l'intéressent pour y revenir plus tard, sans nécessairement passer par une réservation. Les avis publiés par les locataires apportent une couche de confiance précieuse : chaque utilisateur peut noter un logement de une à cinq étoiles et laisser un commentaire textuel. Le tableau de bord personnel centralise toute l'activité de l'utilisateur connecté, qu'il soit propriétaire ou locataire.

L'affichage des annonces peut se faire sous trois formes différentes selon les préférences de chacun. La vue en grille présente des cartes visuelles avec photo, prix et localisation, parfaite pour parcourir rapidement le catalogue. La vue en liste est plus dense et favorise la comparaison. La vue sur carte interactive utilise Leaflet pour afficher la position géographique de chaque annonce, avec un système de clustering qui regroupe les annonces par ville quand on dézoome, et qui les affiche individuellement quand on zoome sur une zone précise. Cette dernière vue est particulièrement utile pour visualiser l'offre disponible dans un quartier.

L'interface complète supporte un mode sombre et un mode clair, persistés dans le navigateur. Ce n'est pas un simple changement de couleurs : tout le système de design adapte ses contrastes pour rester lisible et esthétique dans les deux modes. Une attention particulière a été portée aux animations : un volet de transition glisse à chaque changement de page, les titres importants s'animent lettre par lettre, les compteurs de statistiques s'incrémentent visuellement, et chaque bouton produit une onde au clic. Ces détails apportent une identité forte au site sans nuire à la performance.

2. Architecture technique générale

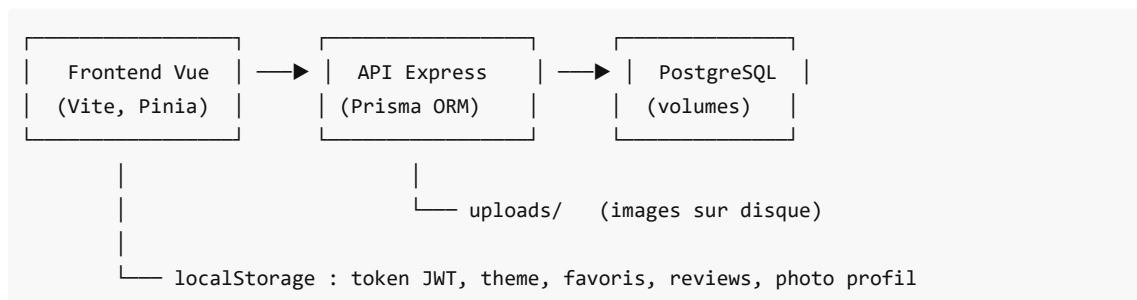
L'application repose sur une **architecture trois tiers** classique, où chaque couche a une responsabilité bien définie. Cette séparation rend le système plus facile à maintenir, à tester et à faire évoluer.

Le **frontend** est une Single Page Application développée en Vue 3 avec Vite. Il s'exécute entièrement dans le navigateur de l'utilisateur et communique avec le backend via des appels REST. Le **backend** est un serveur Express 5 tournant sous Node.js 24. Il expose une API REST documentée par Swagger, gère l'authentification par JWT, et utilise Prisma comme couche d'abstraction pour communiquer avec la base. La **base de données** est PostgreSQL 16, choisie pour sa robustesse et son écosystème mature.

Les images uploadées par les propriétaires sont stockées sur un volume disque persistant côté backend. Cette approche simple suffit pour le projet pédagogique ; en production réelle, on pourrait basculer vers un object store comme S3 ou MinIO.

Côté navigateur, plusieurs informations sont conservées dans le localStorage du client. Le token JWT d'authentification est stocké à la connexion et purgé à la déconnexion ou à l'expiration. La préférence de thème (sombre ou clair) est sauvegardée pour qu'elle persiste entre les sessions. La liste des annonces favorites est synchronisée avec le backend mais également mise en cache localement pour des temps de réponse instantanés. Les avis publiés et la photo de profil de l'utilisateur (encodée en base64) y sont également stockés, ce qui permet une expérience fluide sans aller-retour serveur systématique.

Le développement local s'orchestre avec Docker Compose, qui démarre simultanément la base, l'API, le frontend et une instance PgAdmin pour explorer la base. Cette approche garantit que tous les contributeurs travaillent avec exactement les mêmes versions et qu'un nouveau développeur peut être opérationnel en moins de cinq minutes après avoir cloné le repo. Pour la production, le projet cible un cluster Kubernetes capable de gérer le scaling automatique en cas de pic de trafic, les backups quotidiens de la base, et le monitoring continu de la santé des services. L'intégration continue passe par GitLab CI, qui exécute la suite de tests Jest sur chaque push et déploie automatiquement sur la branche `main`.



3. Frontend Vue 3

3.1 Choix techniques

Le frontend d'ETNAir est une Single Page Application développée avec Vue 3 dans sa syntaxe Composition API moderne (`<script setup>`). Le bundler retenu est Vite, qui offre un démarrage quasi instantané en développement grâce à l'utilisation native des modules ES, ainsi qu'un Hot Module Replacement très réactif. Pour la gestion de l'état global, le projet utilise Pinia, qui remplace l'ancien Vuex dans l'écosystème Vue moderne. Le routing client est assuré par Vue Router en mode history.

Les appels HTTP passent par Axios avec une instance centralisée qui gère automatiquement le token JWT via des intercepteurs. La carte interactive est implémentée avec Leaflet et son binding Vue. Certaines animations utilisent Lottie pour des effets vectoriels légers, comme celle qui s'affiche pendant les transitions de page. Les tests end-to-end sont écrits avec Cypress.

3.2 Organisation du code source

Le code se trouve sous `frontend/src/`. Le point d'entrée est `main.js`, qui initialise Pinia et le router avant de monter l'application sur le DOM. Le composant racine `App.vue` définit le layout global avec la barre de navigation, le pied de page, et un composant `RouterView` qui affiche la vue correspondant à la route active.

Le dossier `views/` contient les onze pages principales de l'application. La page d'accueil présente un carousel d'images, une barre de recherche, et plusieurs sections marketing. La liste des annonces gère les filtres, la pagination, et la bascule entre les vues grille, liste et carte. La fiche détail d'une annonce affiche la galerie d'images, la description, le formulaire de réservation, et la section d'avis. Le tableau de bord utilisateur regroupe trois onglets : les annonces publiées par l'utilisateur (s'il est propriétaire), ses favoris, et son profil personnel. Les formulaires de création et d'édition d'annonces permettent l'upload de plusieurs photos. La page favoris affiche toutes les annonces sauvegardées dans une grille dédiée. Les écrans de connexion et d'inscription incluent une validation stricte de l'email en temps réel. Enfin, les pages institutionnelles "Comment réserver" et "Qui sommes-nous" présentent l'équipe et la démarche.

Les composants réutilisables sont rangés dans `components/`. On y trouve la navbar et le footer bien sûr, mais aussi des éléments transversaux comme `ListingCard` pour afficher une annonce sous forme de carte avec son image, son prix et son bouton favori, `SkeletonCard` pour le placeholder de chargement, `MapView` qui encapsule la carte Leaflet avec son système de clustering par ville, ou encore `ReviewSection` qui gère l'affichage et la création d'avis. Plusieurs composants servent uniquement à l'animation : `PageVolet` pilote la transition entre les pages, `LetterReveal` anime les titres lettre par lettre avec un effet de rotation 3D subtil, et `CountUp` produit des compteurs qui s'incrémentent visuellement.

Les stores Pinia sont rassemblés dans `stores/`. Chacun a une responsabilité claire et un nom explicite. Le store `auth.js` gère l'authentification complète, depuis l'inscription jusqu'à l'expiration automatique du token au bout d'une heure. Le store `theme.js` bascule entre mode clair et mode sombre. Le store `favorites.js` maintient en mémoire les IDs des favoris et leur liste complète. Le store `profilePic.js` permet à chaque utilisateur de stocker sa photo de profil en base64. Enfin, le store `pageTransition.js` synchronise les phases du volet d'animation avec le router.

3.3 Routing et protection des routes

Le fichier `router/index.js` déclare l'ensemble des routes de l'application. Certaines sont totalement publiques : l'accueil, la liste des annonces, la fiche détail, ainsi que les pages "Comment réserver" et "Qui sommes-nous". D'autres nécessitent une authentification active : le dashboard et la page favoris portent l'attribut `meta.auth: true`, ce qui déclenche une redirection vers `/login` si l'utilisateur n'est pas connecté. Les pages d'inscription et de connexion portent à l'inverse l'attribut `meta.guest: true` : si un utilisateur déjà connecté tente d'y accéder, il est renvoyé vers l'accueil.

La création et l'édition d'annonces demandent en plus d'être connecté en tant que propriétaire. Cette vérification se fait au niveau du composant, en utilisant la propriété computed `auth.isOwner` du store. Si un locataire tente d'accéder à ces pages, il est redirigé.

Le router intègre également la gestion du volet d'animation. Dans `beforeEach`, la fonction `coverPage()` est appelée pour déclencher le panneau qui glisse depuis le côté, masquant la vue courante. Une fois le nouveau

composant chargé, `uncoverPage()` est invoquée dans `afterEach` pour faire disparaître le volet. Ce mécanisme garantit que l'animation est parfaitement synchronisée avec le chargement de la nouvelle vue.

3.4 État global avec Pinia

Le store d'authentification est le plus complexe du projet. À l'initialisation, il lit le token éventuellement présent dans le `localStorage` et vérifie sa date d'expiration. Si le token est encore valide, l'utilisateur est considéré comme connecté et ses informations sont immédiatement disponibles. Si l'expiration est dépassée, le store nettoie tout (token, user, expiry) et l'utilisateur est traité comme déconnecté. Une fois logué, un `setTimeout` est programmé pour appeler automatiquement `logout()` au moment précis de l'expiration, ce qui évite à l'utilisateur de rester connecté indéfiniment et limite le risque en cas de vol de token.

Le store de thème est plus simple : il maintient un booléen `dark`, persiste sa valeur dans le `localStorage`, et propage le changement en ajoutant ou retirant l'attribut `data-theme="dark"` sur la balise `<html>`. Toutes les variables CSS sont définies en fonction de cet attribut, ce qui rend la bascule entre les deux modes instantanée et fluide.

Le store des favoris maintient un `Set` des IDs favoris (structure idéale pour des appels `.has(id)` performants), et propose des actions `toggle(id)`, `load()` et `reset()`. Le store de photo de profil est un peu particulier : il s'agit en réalité d'un composable `useProfilePic(userId)`, ce qui permet d'afficher la bonne photo selon l'utilisateur ciblé, qu'il s'agisse de l'utilisateur courant dans la navbar ou d'un autre utilisateur dans une section d'avis.

3.5 Design system

Le fichier `src/assets/main.css` centralise toutes les variables CSS du projet. En mode clair, les couleurs principales sont déclarées sous `:root`. La couleur primaire est un bleu institutionnel `#0458a0`, le fond est un gris très clair `#f9fafb`, le texte est un gris très foncé `#111827`. En mode sombre, ces mêmes variables sont redéfinies sous le sélecteur `[data-theme="dark"]`. La couleur primaire devient blanche, le fond passe à `#18181b` (équivalent à zinc-900 dans le système Tailwind), et plusieurs niveaux de gris sont introduits pour donner de la profondeur visuelle aux différents éléments d'interface.

Les animations apportent une vraie identité au site. Le volet de page glisse horizontalement à chaque navigation, ce qui masque le rechargement du contenu et donne une sensation de fluidité proche d'une vraie application. Les titres importants utilisent `LetterReveal` qui anime chaque caractère avec un léger décalage et un effet de rotation 3D subtil. Les compteurs animés sur les statistiques ajoutent une touche dynamique aux pages d'accueil et de présentation. Au scroll, certains éléments apparaissent grâce à la directive personnalisée `v-reveal` qui utilise un `IntersectionObserver` pour détecter quand un élément entre dans la zone visible. Enfin, chaque bouton génère une onde au clic grâce à un gestionnaire global défini dans `App.vue` qui intercepte tous les clics sur des éléments portant la classe `.btn`.

Chaque vue importe son propre fichier CSS dédié situé dans `src/assets/css/`. Cette organisation permet de garder les composants `.vue` lisibles et de séparer clairement la logique du style.

3.6 Communication avec l'API

Le fichier `services/api.js` contient l'instance Axios partagée par toute l'application. Sa `baseUrl` est configurée à `/api`, ce qui permet à Vite de proxifier les requêtes vers le backend en développement. La cible exacte est lue depuis la variable d'environnement `API_URL`, qui vaut `http://api:3000` dans le contexte Docker Compose.

Un intercepteur de requête ajoute automatiquement le header `Authorization: Bearer <token>` si un token est présent dans le `localStorage`. Cela évite d'avoir à le faire manuellement dans chaque appel. Un intercepteur de réponse gère le cas où le serveur renvoie un 401 : dans ce cas, le token est purgé et l'utilisateur est redirigé vers la page de connexion, ce qui correspond au comportement attendu quand une session a expiré côté serveur.

Les différents endpoints sont regroupés par domaine dans des objets nommés. `authService` regroupe login et register. `listingService` regroupe les annonces avec `getAll`, `getOne`, `create`, `update` et `remove`. `userService` regroupe les utilisateurs. `favoriteService` regroupe les favoris avec `getAll`, `getIds`, `add` et `remove`.

4. Backend Express et API REST

4.1 Choix techniques

Le backend d'ETNAir est un serveur Node.js 24 utilisant le framework Express 5. Le projet est intégralement écrit en modules ECMAScript natifs (`"type": "module"` dans le `package.json`), ce qui apporte une syntaxe d'import moderne mais demande quelques précautions, notamment pour les tests.

L'accès à la base de données passe par Prisma 6, un ORM moderne qui combine une excellente expérience développeur (auto-complétion, types générés) avec des performances solides. Pour PostgreSQL, le projet utilise le nouvel adapter natif `@prisma/adapters-pg` qui remplace l'ancien moteur Rust et offre une meilleure intégration. Les mots de passe sont hashés avec `bcrypt` avant insertion en base, ce qui garantit qu'ils ne sont jamais stockés en clair. Les tokens d'authentification sont des JWT signés avec une clé secrète stockée en variable d'environnement.

Pour la validation des requêtes entrantes, le projet utilise `express-validator` qui permet de déclarer les règles de validation directement dans la définition des routes, avant même que la requête n'atteigne le contrôleur. Les uploads d'images sont gérés par Multer avec un stockage sur disque. La documentation auto-générée est exposée via Swagger UI, alimentée par les commentaires JSDoc présents dans les fichiers de routes. Enfin, les tests sont écrits avec Jest et `supertest` pour la simulation des requêtes HTTP.

4.2 Organisation du code

Le code source est organisé sous `api/src/` selon une architecture classique en couches. Le point d'entrée est `server.js` à la racine, qui configure Express, monte les routes et démarre le serveur sur le port 3000.

Le sous-dossier `controllers/` contient la logique métier de chaque domaine fonctionnel. On y trouve un contrôleur par grande entité : `authController` pour l'inscription et la connexion, `UserController` pour les opérations sur les utilisateurs, `listingController` pour les annonces, `bookingController` pour les réservations, et `favoriteController` pour les favoris. Chaque contrôleur exporte des fonctions qui correspondent aux actions REST (`createBooking`, `getMyBookings`, etc.).

Le sous-dossier `routes/` déclare les routes Express et les associe aux fonctions des contrôleurs. Chaque route applique en amont les middlewares appropriés : la validation des inputs avec `express-validator`, l'authentification avec `authMiddleware` pour les routes protégées, et l'upload de fichiers avec Multer pour les endpoints qui en ont besoin.

Le sous-dossier `middleware/` regroupe les middlewares transversaux. Le fichier `auth.js` contient le middleware qui decode le token JWT depuis le header `Authorization`, vérifie sa validité, charge l'utilisateur correspondant en base, et l'attache à la requête. Le fichier `upload.js` configure Multer avec un stockage sur disque pointant vers `/app/uploads`, applique des filtres sur le type MIME (jpeg, png, gif, webp) et impose une limite de taille à 5 Mo par fichier.

Les utilitaires sont dans `utils/`. Le fichier `hash.js` enveloppe `bcrypt` avec deux fonctions `hashPassword` et `comparePassword`. Le fichier `jwt.js` expose `generateToken(userId)` et `verifyToken(token)` qui utilisent la `JWT_SECRET` lue dans l'environnement.

Le script de seed se trouve dans `scripts/seed.js` et génère les données fictives au premier démarrage. Enfin, les tests sont dans `tests/`, et les mocks Jest dans `__mocks__`.

4.3 Endpoints REST

L'API expose plusieurs domaines fonctionnels, tous accessibles sous le préfixe `/api` après proxification par le frontend.

Pour l'**authentification**, deux endpoints permettent à un utilisateur de créer un compte ou de se connecter. `POST /auth/register` accepte un objet avec un email, un mot de passe (minimum 6 caractères), un prénom et un nom, ainsi qu'un rôle optionnel (`tenant` ou `owner`). La validation est stricte : email au bon format, mot de passe assez long, prénom et nom non vides. Si le compte est créé avec succès, un token JWT est immédiatement renvoyé pour que l'utilisateur soit connecté automatiquement. `POST /auth/login` prend simplement un email et un mot de passe, retourne un token JWT en cas de succès, et un 401 sinon.

Pour les **utilisateurs**, l'endpoint `GET /utilisateurs` retourne la liste complète et ne demande pas d'authentification (utile pour des écrans d'administration). `GET /utilisateurs/:id` exige un token et renvoie le détail d'un utilisateur avec ses annonces associées incluant leurs images. Cet endpoint est utilisé notamment par le dashboard pour afficher les annonces publiées par l'utilisateur courant. `PUT` et `DELETE` permettent respectivement de modifier et supprimer un compte.

Pour les **annonces**, l'endpoint principal `GET /annonces` retourne la liste paginée. Il accepte des paramètres query comme `page`, `limit`, `city` et `maxPrice` pour filtrer les résultats. Chaque annonce retournée contient son propriétaire, ses images, et un flag `isBooked` qui indique si une réservation confirmée et active existe actuellement. Ce flag est calculé directement par Prisma via un `_count` qui compte les bookings respectant les conditions `status: 'confirmed'` et `endDate: { gte: now }`.

`GET /annonces/:id` retourne le détail complet d'une annonce. `POST /annonces` est protégé et permet à un propriétaire de créer une nouvelle annonce. La requête est de type `multipart/form-data` pour pouvoir inclure jusqu'à cinq images. `PUT /annonces/:id` gère aussi bien l'ajout de nouvelles images que la suppression d'images existantes via un paramètre `removeImageIds`. Enfin `DELETE /annonces/:id` supprime l'annonce, ce qui déclenche en cascade la suppression de ses images, bookings et avis associés.

Pour les **favoris**, quatre endpoints sont disponibles. `GET /favoris` renvoie la liste complète des annonces favorites de l'utilisateur courant avec toutes leurs informations. `GET /favoris/ids` est plus léger : il renvoie uniquement les IDs, ce qui est utilisé par le frontend pour savoir quelles cartes afficher avec un cœur rouge sans avoir à charger toutes les données. `POST /favoris/:listingId` ajoute une annonce, et `DELETE /favoris/:listingId` la retire. Tous ces endpoints requièrent une authentification.

Pour les **réservations**, `POST /reservations` permet à un locataire de créer une demande. Le contrôleur vérifie plusieurs choses avant d'enregistrer : que les dates demandées ne sont pas dans le passé, qu'il y a au moins une nuit, que l'utilisateur n'est pas le propriétaire de sa propre annonce, et qu'aucune autre réservation en attente ou confirmée ne chevauche les dates demandées. Si tout est bon, la réservation est créée avec le statut `pending` et un `cancelDeadline` calculé à 48h avant la date d'arrivée. Les endpoints `GET /reservations/mes-reservations` et `GET /reservations/recues` permettent respectivement à un locataire de voir ses propres résas et à un propriétaire de voir les résas qu'il a reçues. `PUT /reservations/:id/confirmer` permet au propriétaire d'accepter une demande, et `PUT /reservations/:id/annuler` peut être appelée par les deux parties tant que le délai n'est pas dépassé.

4.4 Authentification et sécurité

Le flux d'authentification suit le modèle JWT classique. Lors d'un login réussi, le serveur génère un token avec le `userId` comme payload et le retourne au frontend, qui le stocke dans le `localStorage`. À chaque requête ultérieure, l'intercepteur `Axios` ajoute automatiquement ce token dans le header `Authorization: Bearer <token>`. Le `authMiddleware` du backend extrait le token, le décode avec `verifyToken`, charge l'utilisateur correspondant en base via `prisma.user.findUnique`, et attache son ID à `req.userId` pour que les contrôleurs puissent l'utiliser sans avoir à refaire la vérification.

Les mots de passe sont hashés avec `bcrypt` en utilisant 10 rounds, ce qui est un bon compromis entre sécurité et performance. Les comparaisons utilisent `bcrypt.compare` qui est sécurisé contre les timing attacks. La `JWT_SECRET` est injectée en variable d'environnement et ne doit jamais être committée dans le repo. Bien que le token n'ait pas d'expiration native côté serveur, le frontend impose une expiration de 1 heure côté client, ce qui force l'utilisateur à se reconnecter régulièrement et limite la fenêtre d'exposition en cas de vol de token.

4.5 Gestion des uploads

Le middleware `upload.js` configure Multer avec un stockage sur disque (`multer.diskStorage`). Le dossier de destination est `/app/uploads` dans le conteneur, monté en volume Docker vers `./api/uploads` sur l'hôte. Cela garantit que les images persistent entre les redémarrages et même entre les rebuilds de l'image Docker. Les noms de fichiers sont préfixés par un timestamp et un suffixe aléatoire pour éviter les collisions, par exemple `1719987234567-abc123xyz.jpg`.

Un filtre MIME accepte uniquement les formats `jpeg`, `jpg`, `png`, `gif` et `webp`, ce qui évite qu'un utilisateur malveillant n'uploadé un exécutable déguisé. La taille de chaque fichier est limitée à 5 Mo, et un maximum de 5 fichiers peut être envoyé dans une seule requête. Les images sont ensuite servies de manière statique via `app.use('/uploads', express.static(...))`, ce qui les rend accessibles publiquement via leur URL.

5. Base de données PostgreSQL et Prisma

5.1 Choix techniques

La base de données d'ETNAir est une instance PostgreSQL 16, choisie pour sa robustesse, ses fonctionnalités avancées et son large écosystème. L'accès au SGBD passe exclusivement par Prisma 6, un ORM moderne qui combine excellente expérience développeur et performances solides. Le projet n'écrit volontairement aucune requête SQL brute : toute la couche d'accès aux données passe par les méthodes typées de Prisma. Cela garantit que le schéma reste cohérent entre le code et la base, et que les migrations sont gérées de manière déclarative via le fichier `schema.prisma`.

5.2 Modèle de données

Le modèle relationnel articule six entités principales qui correspondent aux concepts métier de la plateforme. Un utilisateur peut être soit locataire (`tenant`), propriétaire (`owner`), ou administrateur (`admin`). Un propriétaire peut publier plusieurs annonces, chacune comportant des images, pouvant faire l'objet de réservations, recevant des avis, et étant éventuellement mise en favori par d'autres utilisateurs.

L'entité `User` représente n'importe quel utilisateur de la plateforme. Elle contient les informations classiques : un identifiant auto-incrémenté, un prénom, un nom, une adresse email unique, un mot de passe hashé via `bcrypt` et un rôle. Une date de création est automatiquement renseignée. Côté relations, un utilisateur peut avoir des annonces s'il est propriétaire, des réservations s'il est locataire, des favoris et des avis dans tous les cas.

L'entité `Listing` représente une annonce de logement. Elle contient un titre, une description, une ville, un prix par nuit en `Decimal` avec deux chiffres après la virgule pour éviter les erreurs de précision flottante, des coordonnées GPS optionnelles, et des dates de disponibilité. Chaque annonce est rattachée à un propriétaire via la clé étrangère `ownerId`, avec une suppression en cascade : si le propriétaire supprime son compte, ses annonces sont supprimées.

L'entité `ListingImage` stocke les images uploadées. Chaque ligne contient un `url` qui est en réalité un chemin relatif vers `/uploads/<filename>`, une `key` qui correspond au nom du fichier sur disque, et un `listingId` qui fait le lien avec l'annonce. La suppression cascade garantit que les références en base sont nettoyées quand une annonce est supprimée.

L'entité `Booking` enregistre une demande de réservation. Elle stocke l'utilisateur locataire, l'annonce concernée, les dates de début et de fin, le prix total calculé, et un statut qui prend l'une des valeurs `pending`, `confirmed` ou `cancelled`. Un champ `cancelDeadline` est calculé automatiquement à la création (48 heures avant la date d'arrivée) et sert de référence pour autoriser ou refuser une annulation tardive.

L'entité `Favorite` est une simple table de jointure entre `User` et `Listing`, avec une contrainte d'unicité qui empêche un utilisateur de mettre la même annonce en favori plusieurs fois. La suppression cascade dans les deux sens garantit la cohérence référentielle.

L'entité `Review` représente un avis laissé sur une annonce. Elle contient une note de 1 à 5, un commentaire textuel optionnel, et les références à l'utilisateur auteur et à l'annonce concernée. À noter : sur le frontend actuel, les avis sont stockés en `localStorage` pour la démo, le modèle existe en base mais n'est pas encore branché à des endpoints REST.

5.3 Migrations

Toutes les migrations sont stockées dans `api/prisma/migrations/` sous forme de fichiers SQL générés par Prisma. La migration initiale crée les tables `User`, `Listing`, `Booking` et `Review`. Une migration ultérieure ajoute les champs `latitude` et `longitude` au modèle `Listing` pour permettre l'affichage sur la carte. Une autre crée la table `ListingImage` (auparavant les images étaient stockées sur MinIO). Enfin, une dernière crée la table `Favorite`.

Pour appliquer toutes les migrations en attente sur une base existante, il suffit d'exécuter `docker compose exec api npx prisma migrate deploy`. Cette commande est idempotente et ne réapplique pas les migrations déjà passées. Elle est aussi appelée automatiquement dans le `docker-entrypoint.sh` à chaque démarrage du conteneur, ce qui garantit que le schéma de la base est toujours synchronisé avec le code.

5.4 Script de seed

Le fichier `api/src/scripts/seed.js` peuple la base avec des données réalistes au premier démarrage. Le mécanisme repose sur un utilisateur sentinelle dont l'email est `seed@etnair.com`. Si cet utilisateur existe déjà au démarrage, le script considère que la base a déjà été seedée et se contente d'afficher un message avant de rendre la main. Sinon, il procède au peuplement.

Le seed génère d'abord 35 utilisateurs (25 propriétaires et 10 locataires) avec des prénoms, noms et emails fictifs grâce à la bibliothèque `@faker-js/faker`. Tous ces comptes ont le même mot de passe `password123`, ce qui facilite les tests manuels. Ensuite, le script crée environ 127 annonces réparties dans 35 villes françaises. Chaque ville a ses coordonnées GPS de référence, et chaque annonce reçoit ces coordonnées avec un léger jitter aléatoire pour éviter que toutes les annonces d'une même ville se superposent sur la carte. Les titres sont construits à partir d'une liste d'adjectifs et de types de logements pour donner des résultats du style "Charmant Studio à Lyon" ou "Lumineux T2 à Bordeaux".

6. Conteneurisation avec Docker

6.1 Présentation

L'environnement de développement d'ETNAir repose entièrement sur Docker Compose, ce qui permet à n'importe quel développeur de démarrer le projet complet en une seule commande, sans avoir à installer manuellement Node.js, PostgreSQL ou PgAdmin sur sa machine. Cette approche garantit aussi que tous les contributeurs travaillent avec exactement les mêmes versions des dépendances et des services, ce qui élimine les classiques "ça marche chez moi".

Cinq services tournent en parallèle sur un réseau Docker isolé. Le frontend est exposé sur le port 5173 et sert l'application Vue via le dev server de Vite. L'API Express tourne sur le port 3000. La base PostgreSQL écoute sur le port 5432 (mappé en 5433 côté hôte pour éviter les conflits avec une éventuelle instance locale). Une instance PgAdmin est disponible sur le port 5050 pour explorer la base via une interface graphique. Tous ces services sont reliés par un réseau bridge nommé `etnaïr-network`, ce qui leur permet de communiquer entre eux en utilisant leurs noms de service plutôt que des adresses IP.

6.2 Structure du docker-compose.yml

Le fichier `docker-compose.yml` à la racine du projet déclare l'ensemble des services. Le service `db` utilise l'image officielle `postgres:16` et déclare trois variables : `POSTGRES_USER`, `POSTGRES_PASSWORD` et `POSTGRES_DB`, qui sont automatiquement consommées par l'image pour créer la base et le compte au premier démarrage. Un volume nommé `db_data` est monté pour que les données persistent entre les redémarrages.

Le service `api` est construit à partir du `Dockerfile` situé dans `./api`. Il déclare son `DATABASE_URL` complet, son `JWT_SECRET`, et son `PORT`. La directive `depends_on: [db]` garantit que PostgreSQL démarre avant l'API. Un bind mount permet aux images uploadées d'être visibles depuis l'hôte, ce qui est pratique pour les inspecter mais surtout pour qu'elles survivent aux rebuilds de l'image. La politique `restart: unless-stopped` redémarre automatiquement le conteneur en cas de crash.

Le service `frontend` est lui aussi buildé depuis son `Dockerfile`. Il reçoit la variable `API_URL=http://api:3000` qui est utilisée par le proxy Vite pour rediriger les appels `/api/*` vers le backend. Un bind mount permet le hot reload pendant le développement : toute modification de fichier sur l'hôte est immédiatement visible dans le conteneur. Un volume anonyme sur `/app/node_modules` est crucial car il empêche le bind mount d'écraser les `node_modules` installés dans l'image (qui sont parfois différents selon la plateforme).

6.3 Commandes courantes

Pour démarrer toute la stack, une seule commande à la racine du projet suffit : `docker compose up -d`. Le flag `-d` lance les services en arrière-plan. Au premier lancement, Docker doit télécharger les images et builder l'API et le frontend, ce qui prend quelques minutes. Les lancements suivants sont quasi instantanés.

Pour voir l'état de tous les conteneurs, on utilise `docker compose ps`. Pour suivre les logs d'un service en temps réel, c'est `docker compose logs -f api`. Cette commande est très utile pour diagnostiquer un démarrage qui se passe mal.

Quand on modifie le `Dockerfile` ou le `package.json` d'un service, il faut reconstruire l'image avec `docker compose build api`, puis relancer le service avec `docker compose up -d api`. À l'inverse, si on modifie juste du code source, pas besoin de rebuild : le hot reload prend le relais grâce au bind mount.

Pour tout reset, y compris la base de données, la commande est `docker compose down -v`. Le `-v` supprime aussi les volumes, ce qui efface complètement la base. Cette commande est utile quand on veut tester le seed depuis zéro.

Enfin, pour exécuter une commande dans un conteneur sans avoir à entrer dedans, on utilise `docker compose exec`. Par exemple `docker compose exec api npm test` lance les tests Jest. `docker compose exec db psql -U etnair_user -d etnair_db` ouvre une console PostgreSQL.

7. Déploiement Kubernetes

7.1 Présentation

Là où Docker Compose suffit largement pour le développement local, la production d'ETNAir cible un cluster Kubernetes. Le passage à Kubernetes apporte trois bénéfices majeurs. D'abord, la haute disponibilité : plusieurs replicas de chaque service tournent en parallèle, et si un pod crash le service reste accessible grâce aux autres. Ensuite, le scaling automatique : un HorizontalPodAutoscaler ajuste le nombre de replicas en fonction de la charge réelle. Enfin, la gestion déclarative : tout l'état du cluster est décrit dans des manifests YAML versionnables, ce qui permet de reproduire l'environnement à l'identique sur n'importe quel cluster.

Les manifests sont rassemblés dans le dossier `k8s/` à la racine du repo. Le pipeline GitLab CI applique automatiquement ces manifests sur la branche `main` après avoir buildé et poussé les nouvelles images Docker sur Docker Hub.

7.2 Architecture sur le cluster

Un ingress controller (typiquement nginx-ingress) reçoit tout le trafic public sur le port 443. Il route les requêtes selon le chemin demandé : tout ce qui commence par `/api/` est envoyé vers le service Kubernetes `api`, et tout le reste est envoyé vers le service `frontend`. Le service `api` répartit la charge entre plusieurs pods (entre 2 et 10 selon la charge), tandis que le service `frontend` distribue vers deux pods statiques.

Les pods d'API communiquent avec un service `postgres` qui pointe vers le pod PostgreSQL backé par un volume persistant. Enfin, un CronJob lance chaque nuit un job de backup qui dump la base et l'écrit sur un PVC dédié.

7.3 HorizontalPodAutoscaler

Le fichier `hpa.yml` configure l'autoscaling de l'API. La règle est simple : si la consommation CPU moyenne dépasse 70% pendant trois minutes, Kubernetes ajoute un nouveau pod, et continue jusqu'à un maximum de 10 replicas. Inversement, si la charge baisse durablement, des pods sont supprimés progressivement jusqu'au minimum de 2 replicas.

Le seuil de 70% est un compromis classique : un seuil plus bas déclencherait du scaling plus tôt mais consommerait plus de ressources pour pas grand-chose en cas de pics ponctuels. Un seuil plus haut serait plus économe mais risquerait de saturer les pods avant que les nouveaux ne soient prêts.

Pour observer l'autoscaler en action, on peut lancer une session de monitoring continu avec `kubectl get hpa -w` et `kubectl get pods -w` dans deux terminaux séparés, puis déclencher un test de charge depuis un autre poste avec k6. On voit alors le CPU monter, les replicas augmenter progressivement jusqu'à atteindre un palier qui absorbe la charge, puis redescendre quelques minutes après la fin du test.

7.4 Sécurité et secrets

Les variables sensibles ne sont jamais hardcodées dans les manifests YAML. Au lieu de cela, on crée un Secret Kubernetes qui contient `JWT_SECRET`, `POSTGRES_PASSWORD` et autres valeurs critiques. Les Deployments référencent ensuite ces secrets via `secretKeyRef`, ce qui fait que Kubernetes injecte la bonne valeur dans la variable d'environnement du conteneur au démarrage. L'avantage est que les secrets ne se retrouvent jamais dans le repo Git, ils restent stockés uniquement dans etcd côté cluster.

7.5 Backups automatiques

Le CronJob défini dans `backup-cronjob.yml` s'exécute chaque nuit à 2h du matin. Il lance un conteneur PostgreSQL temporaire qui se connecte à la base de production, exécute `pg_dump` pour exporter toute la base, compresse le résultat avec gzip, et l'écrit sur un PVC dédié aux backups. Le nommage des fichiers inclut la date du jour, ce qui facilite la recherche d'un backup spécifique.

8. Stratégie de tests

8.1 Présentation

La stratégie de test d'ETNAir s'articule autour de trois niveaux complémentaires qui couvrent des aspects différents de la qualité. Les tests unitaires Jest valident la logique métier du backend en isolant chaque contrôleur. Les tests end-to-end Cypress reproduisent le comportement réel d'un utilisateur dans le navigateur. Et les tests de charge k6 vérifient la résilience de l'infrastructure sous pression.

Cette approche pyramidale est volontaire : on écrit beaucoup de tests unitaires (rapides, ciblés, faciles à maintenir), un nombre raisonnable de tests E2E (plus lents mais plus représentatifs du vécu utilisateur), et seulement quelques tests de charge (coûteux en ressources mais essentiels pour la production).

8.2 Tests unitaires Jest

Pour exécuter la suite de tests backend, on se place dans le dossier `api/` et on lance `npm test`. Cette commande exécute Jest avec la configuration ESM nécessaire, génère un rapport de couverture, et affiche les résultats.

L'approche choisie est de tout mocker au niveau Prisma, ce qui évite d'avoir besoin d'une vraie base de données pour exécuter les tests. Un fichier `src/__mocks__/prisma.js` exporte un objet `prisma` factice où chaque méthode est une `jest.fn()` configurable test par test. Le mock est appliqué automatiquement grâce à `moduleNameMapper` dans `jest.config.js`. Toute importation de `lib/prisma.js` est redirigée vers le mock, que ce soit depuis un contrôleur, un middleware ou un test.

Comme le projet utilise ESM nativement, Jest doit être lancé avec `node --experimental-vm-modules`, et les imports `jest`, `describe`, `it` ne sont pas globaux : ils doivent être importés explicitement depuis `@jest/globals`.

La suite de tests actuelle couvre quatre domaines. Le fichier `auth.test.js` vérifie que l'inscription accepte un email valide mais rejette les emails malformés ou les mots de passe trop courts, et que le login retourne un token avec les bons identifiants. Le fichier `utilisateurs.test.js` teste les endpoints CRUD avec authentification. Le fichier `annonces.test.js` couvre la pagination, le filtre par ville, la création et la suppression. Enfin `booking.test.js` valide les flux de réservation, de confirmation, d'annulation et la disponibilité, et vérifie que le flag `isBooked` reflète correctement l'état de la base.

8.3 Tests end-to-end Cypress

Les tests E2E se trouvent dans `frontend/cypress/e2e/`. Pour les lancer en local, on a deux options. Le mode interactif s'invoque par `npm run cy:open` : il ouvre une interface graphique où on choisit le fichier de spécifications à exécuter, et on voit le navigateur s'animer en temps réel. Le mode headless s'invoque par `npm run cy:run` : aucune interface, juste les résultats dans le terminal.

Un choix architectural fort a été fait : les appels API sont systématiquement interceptés avec `cy.intercept()` et alimentés par des fixtures JSON. Cela rend les tests E2E indépendants du backend : ils peuvent tourner même si l'API n'est pas démarrée, ce qui simplifie énormément l'exécution en CI.

Les commandes custom définies dans `cypress/support/commands.js` simplifient les tests répétitifs.

`cy.login(email, password)` effectue un login via API. `cy.logout()` nettoie le localStorage.

`cy.mockListings()`, `cy.mockListing(id)` et `cy.mockFavorites()` configurent les interceptions courantes.

Le fichier `auth.cy.js` teste l'inscription et la connexion avec des identifiants valides et invalides, la déconnexion, ainsi que la protection des routes privées. Le fichier `annonces.cy.js` vérifie l'affichage de la liste, le filtre par ville, les différentes vues, et la navigation vers une fiche détail. Le fichier `dashboard.cy.js` couvre l'accès au tableau de bord, l'affichage des onglets et la suppression. Enfin `favoris.cy.js` teste l'ajout et le retrait des favoris.

8.4 Tests de charge k6

Les tests de charge sont écrits avec k6, un outil moderne en Go qui exécute des scripts JavaScript pour simuler du trafic. Le script principal est `k6/load-test.js`. Il définit un profil de charge en trois phases : montée progressive jusqu'à 10 utilisateurs virtuels sur 30 secondes, plateau à 10 VUs pendant une minute, puis descente à 0 sur 30 secondes. Deux seuils de qualité sont définis : 95% des requêtes doivent répondre en moins de 500ms, et le taux d'erreurs doit rester inférieur à 1%.

Pour lancer le test, il faut d'abord installer k6 sur sa machine (`choco install k6` sur Windows, `brew install k6` sur Mac). Ensuite, la commande est simplement `k6 run k6/load-test.js`.

L'intérêt majeur du test de charge est de valider le comportement du HorizontalPodAutoscaler Kubernetes. Si on lance k6 avec un nombre élevé de VUs, on peut observer en parallèle les replicas de l'API monter automatiquement, puis redescendre quelques minutes après la fin du test. C'est la démonstration concrète que l'infrastructure est résiliente face à des pics de trafic.

9. Pipeline CI/CD GitLab

Le fichier `.gitlab-ci.yml` à la racine du projet définit le pipeline d'intégration continue. Trois stages sont déclarés : `test`, `e2e` et `deploy`.

Le stage **test** utilise l'image `node:24-alpine`, installe les dépendances de l'API, et lance `npm test`. L'option `|| true` est ajoutée à la fin de la commande pour éviter que le pipeline ne tombe en cas de tests qui échouent. Cette décision est volontaire mais discutable : elle a été prise parce que la VM GitLab a peu de RAM et que certains tests Jest pouvaient timeout. Pour une vraie mise en production, il faudrait retirer ce `|| true` et corriger la cause racine des échecs.

Le stage **e2e** déclenche les tests Cypress. Il utilise l'image `cypress/Included:13.6.0` qui fait environ 2 GB et contient déjà Chromium pré-installé. Comme la VM de la CI a seulement 4 GB de RAM, ce stage est configuré en `when: manual`, c'est-à-dire qu'il ne s'exécute jamais automatiquement mais nécessite un clic dans l'interface GitLab. Cette contrainte permet quand même de démontrer que les tests existent et sont exécutables.

Le stage **deploy** ne s'exécute que sur la branche `main`. Il construit les images Docker de l'API et du frontend, les pousse sur Docker Hub avec le tag `latest`, télécharge `kubect1`, configure l'accès au cluster via une variable

d'environnement `KUBECONFIG` , et applique tous les manifests Kubernetes du dossier `k8s/` . Le résultat est qu'un push sur `main` déclenche un déploiement complet en production en quelques minutes.

10. Démarrage rapide et commandes utiles

10.1 Premier lancement

Pour démarrer ETNAir en local, il suffit d'avoir Docker Desktop installé puis d'exécuter trois commandes. D'abord, cloner le dépôt et entrer dans le dossier. Ensuite, lancer toute la stack avec `docker compose up -d` . Au bout d'une trentaine de secondes, tous les services sont disponibles.

Le frontend est accessible sur `http://localhost:5173` , l'API sur `http://localhost:3000` , et sa documentation Swagger sur `http://localhost:3000/api-docs` . Une interface d'administration PgAdmin est également disponible sur `http://localhost:5050` (login `harvey@etnair.xyz` , mot de passe `harvey`).

Au premier démarrage, un script de seed peuple automatiquement la base avec 35 utilisateurs fictifs et environ 127 annonces réparties dans 35 villes françaises, ce qui permet d'avoir immédiatement un dataset réaliste pour tester l'interface.

10.2 Comptes de test

Le seed crée plusieurs comptes utilisables pour tester l'application. Tous ces comptes ont le même mot de passe `password123` . Pour tester en tant que propriétaire, on peut utiliser l'un des 25 comptes générés automatiquement (visibles dans la liste des utilisateurs ou en interrogeant la base). Pour tester en tant que locataire, on peut utiliser l'un des 10 comptes de locataires générés. Bien sûr, on peut aussi créer un nouveau compte directement via la page d'inscription.

10.3 Commandes utiles

Pour redémarrer un service après une modification de configuration : `docker compose restart api` . Pour rebuild une image après modification du `Dockerfile` ou du `package.json` : `docker compose build api && docker compose up -d api` . Pour tout effacer et repartir d'une base vide : `docker compose down -v && docker compose up -d` .

Pour exécuter les tests backend : `cd api && npm test` . Pour ouvrir Cypress en mode interactif : `cd frontend && npm run cy:open` . Pour lancer un test de charge k6 sur l'API distante : `k6 run k6/load-test.js` .

Pour accéder à la base PostgreSQL en ligne de commande : `docker compose exec db psql -U etnair_user -d etnair_db` . Pour appliquer manuellement les migrations Prisma : `docker compose exec api npx prisma migrate deploy` . Pour forcer un re-seed de la base, il faut d'abord supprimer l'utilisateur sentinelle puis redémarrer l'API.

Conclusion

ETNAir est un projet complet qui couvre l'ensemble du cycle de développement d'une application web moderne, depuis la conception du modèle de données jusqu'au déploiement automatisé en production. L'architecture trois tiers classique reste robuste et bien comprise, mais elle est ici enrichie par des choix techniques contemporains (Vue 3 Composition API, ESM natif, Prisma, Kubernetes) qui rendent le code lisible et maintenable.

Les fonctionnalités utilisateur couvrent les attentes d'une plateforme de mise en relation : authentification sécurisée, recherche avancée, géolocalisation, favoris, avis, réservations avec gestion des conflits de dates. L'interface elle-même propose une expérience travaillée avec mode sombre, animations soignées et responsive design.

Côté infrastructure, le projet démontre la maîtrise des outils modernes du DevOps : conteneurisation avec Docker, orchestration avec Kubernetes, scaling automatique via HPA, backups planifiés, monitoring avec Prometheus et Grafana, pipeline CI/CD avec GitLab. La stratégie de tests à trois niveaux (unitaires, end-to-end, charge) garantit la qualité et la résilience.

Les axes d'amélioration identifiés constituent autant de pistes pour faire évoluer le projet : ajout de tests d'intégration avec une vraie base, implémentation des endpoints d'avis, chaos engineering pour valider la tolérance aux pannes, et passage à un object store pour les images en production. Ces évolutions sont réalistes et démontrent qu'au-delà de l'état actuel du code, l'équipe a une vision claire de la maturité technique à atteindre.